

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

SIMPLE E-COMMERCE STORE

Project Type	Full-Stack Web Application
Frontend	HTML, CSS, JavaScript
Backend	Python, Django, Django REST Framework
Database	SQLite / MySQL
Architecture	Client–Server / Three-Tier Architecture
API Communication	REST API with JSON over HTTP

This document defines the functional requirements, non-functional requirements, system architecture, database design, REST API requirements, data flow, security requirements, and user interface requirements for the Simple E-Commerce Store.

1. INTRODUCTION

1.1 Purpose

The purpose of the Simple E-Commerce Store is to develop a web-based online shopping platform that allows customers to browse products, view product details, add products to a shopping cart, place orders, and view their order history. The system also provides an Administrator module for managing products, categories, stock, and customer orders.

1.2 Scope

The system consists of two major user roles: Customer and Administrator.

Customer Module

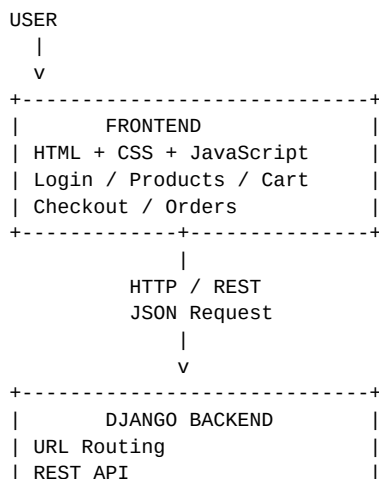
- User registration and login/logout
- Product browsing, searching and filtering
- Product details
- Shopping cart and quantity management
- Checkout and order placement
- Order confirmation
- Order history and order status tracking

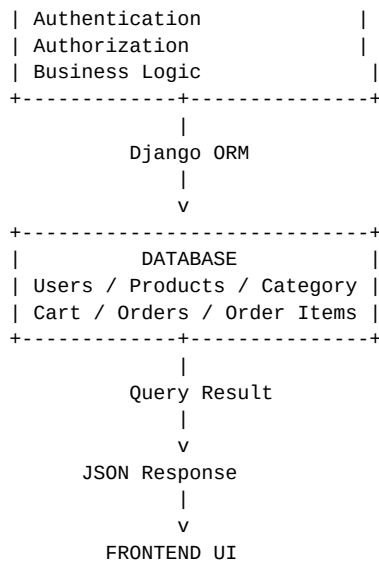
Administrator Module

- Admin login
- Add, update and delete products
- Manage categories
- Manage stock
- View customer orders
- Update order status

2. SYSTEM OVERVIEW

The system follows a Three-Tier Client-Server Architecture. The presentation layer contains the HTML/CSS/JavaScript frontend, the application layer contains Django and Django REST Framework, and the data layer contains SQLite or MySQL.





3. SYSTEM ARCHITECTURE

3.1 Presentation Layer – Frontend

- HTML for page structure
- CSS for styling and responsive design
- JavaScript for client-side validation, API calls and dynamic UI updates

3.2 Application Layer – Backend

- Python and Django
- Django REST Framework for REST APIs
- Authentication and authorization
- Business logic and validation
- Order processing and stock verification

3.3 Data Layer – Database

- Users
- Products
- Categories
- Cart and Cart Items
- Orders and Order Items

4. DATA FLOW ARCHITECTURE

The core communication pattern is: User → Frontend → HTTP Request → Django REST API → Business Logic → Django ORM → Database → JSON Response → Frontend → User.



```

Django REST API
|
v
Business Logic
|
v
Django ORM
|
v
Database
|
| Query Result
v
Django Backend
|
| JSON Response
v
JavaScript
|
v
Updated UI
|
v
USER

```

5. FUNCTIONAL REQUIREMENTS

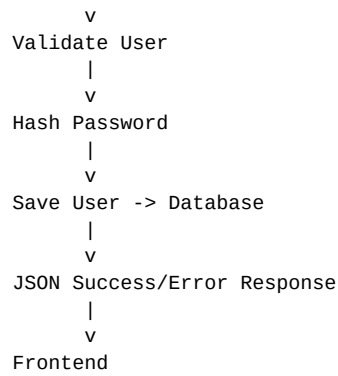
ID	Requirement	Description
FR-01	User Registration	Allow a new customer to create an account using name, email and password.
FR-02	User Login	Authenticate registered users and protect restricted features.
FR-03	Product Listing	Display available products with image, name, price, category and stock.
FR-04	Product Details	Display complete information for a selected product.
FR-05	Search and Filter	Allow searching by product name and filtering by category, price and availability.
FR-06	Shopping Cart	Add, update quantity, remove items and calculate totals.
FR-07	Checkout	Collect delivery details and show an order summary before confirmation.
FR-08	Order Processing	Validate user, stock and total; create order and order items; reduce stock; clear cart.
FR-09	Order Status	Track order through PLACED, CONFIRMED, PROCESSING, SHIPPED, DELIVERED or CANCELLED.
FR-10	Order History	Allow authenticated users to view their previous orders.
FR-11	Admin Product Management	Provide CRUD operations for products.
FR-12	Stock Management	Allow administrators to manage and monitor stock.
FR-13	Order Management	Allow administrators to view orders and update order status.

5.1 Registration Data Flow

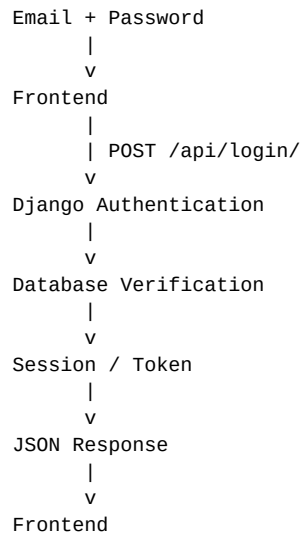
```

Registration Form
|
v
Frontend Validation
|
| POST /api/register/
v
Django API
|

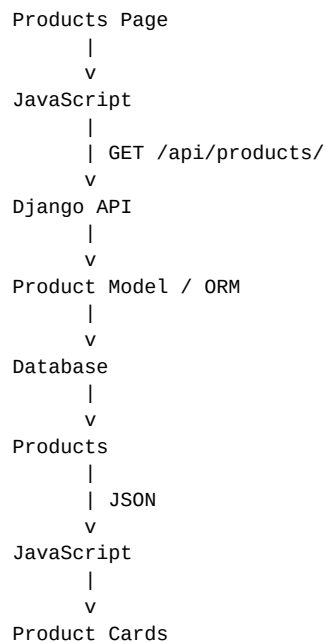
```



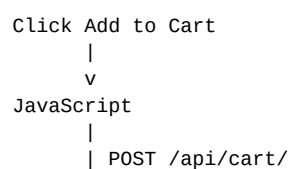
5.2 Login Data Flow

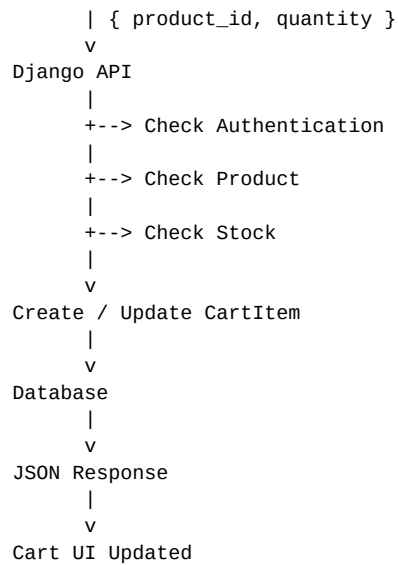


5.3 Product Listing Data Flow

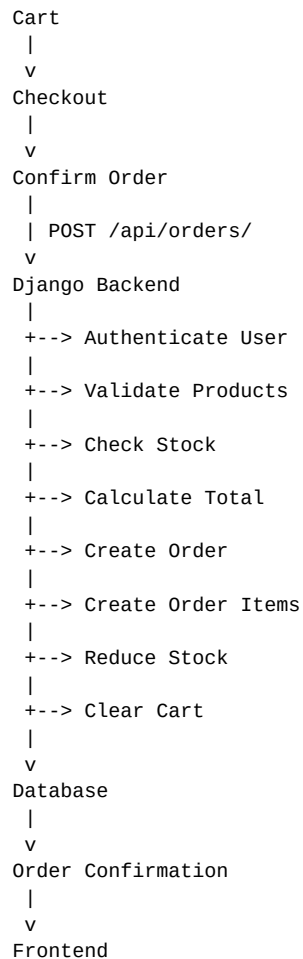


5.4 Add-to-Cart Data Flow





5.5 Order Processing Data Flow



6. DATABASE REQUIREMENTS

6.1 User Table

Field	Description
id	Unique user ID

Field	Description
username	User name
email	User email
password	Hashed password
created_at	Account creation date

6.2 Category Table

Field	Description
id	Category ID
name	Category name
description	Category description

6.3 Product Table

Field	Description
id	Product ID
name	Product name
description	Product description
price	Product price
image	Product image
stock	Available quantity
category_id	Category reference
created_at	Creation date
updated_at	Last update

6.4 Cart Table

Field	Description
id	Cart ID
user_id	User reference
created_at	Cart creation date

6.5 Cart Item Table

Field	Description
id	Cart item ID

Field	Description
cart_id	Cart reference
product_id	Product reference
quantity	Selected quantity

6.6 Order Table

Field	Description
id	Order ID
user_id	Customer reference
total_amount	Total order value
shipping_address	Delivery address
phone	Customer phone
status	Order status
created_at	Order date

6.7 Order Item Table

Field	Description
id	Order item ID
order_id	Order reference
product_id	Product reference
quantity	Ordered quantity
price	Price at purchase time

6.8 Database Relationships

```

USER 1 ----- 1 CART
CART 1 ----- * CART_ITEM
CATEGORY 1 ---- * PRODUCT
USER 1 ----- * ORDER
ORDER 1 ----- * ORDER_ITEM
PRODUCT 1 ----- * ORDER_ITEM
PRODUCT 1 ----- * CART_ITEM

```

The OrderItem price field stores the price at the time of purchase so that historical orders remain accurate even if the product price changes later.

7. REST API REQUIREMENTS

7.1 Authentication APIs

Method	Endpoint	Purpose
POST	/api/register/	Register user
POST	/api/login/	Login
POST	/api/logout/	Logout

7.2 Product APIs

Method	Endpoint	Purpose
GET	/api/products/	Get all products
GET	/api/products/{id}/	Get product details
POST	/api/products/	Add product (Admin)
PUT	/api/products/{id}/	Update product (Admin)
DELETE	/api/products/{id}/	Delete product (Admin)

7.3 Cart APIs

Method	Endpoint	Purpose
GET	/api/cart/	View cart
POST	/api/cart/	Add item
PUT	/api/cart/{id}/	Update quantity
DELETE	/api/cart/{id}/	Remove item

7.4 Order APIs

Method	Endpoint	Purpose
POST	/api/orders/	Place order
GET	/api/orders/	Order history
GET	/api/orders/{id}/	Order details
PUT	/api/orders/{id}/	Update status (Admin)

8. NON-FUNCTIONAL REQUIREMENTS

8.1 Performance

- API requests should be processed efficiently.
- Product listing should load within a reasonable time.
- Database queries should be optimized.

8.2 Security

- Passwords must be securely hashed.
- Authentication must be implemented.
- Protected APIs must require authentication.
- Admin operations must require authorization.
- User input must be validated.
- Users must only access their own orders.

8.3 Usability

- The interface should be simple and user-friendly.
- Navigation should be clear.
- Error messages should be understandable.
- The application should be responsive on different screen sizes.

8.4 Reliability

- Invalid orders must not be created.
- Products with insufficient stock must not be ordered.
- Order and stock information must remain consistent.

8.5 Maintainability

The application should use a modular structure so that frontend, backend, APIs and database components can be modified independently.

9. ERROR HANDLING

HTTP Code	Meaning
200	Successful request
201	Resource created
400	Bad request / invalid input
401	Unauthorized
403	Forbidden
404	Resource not found
500	Internal server error

```

Example:
Invalid Login
  |
  v
Django
  |
  v
HTTP 401
  |
  v
JSON Error
  |
  v

```

Frontend
|
v
"Invalid email or password"

10. USER INTERFACE REQUIREMENTS

10.1 Home Page

- Store logo/name
- Navigation bar
- Search bar
- Product categories
- Featured products
- Login/Register
- Cart icon

10.2 Products Page

- Product cards
- Product image
- Product name
- Price
- Category
- Stock
- View Details
- Add to Cart

10.3 Product Details Page

- Large product image
- Product name
- Description
- Price
- Stock
- Quantity selector
- Add to Cart

10.4 Cart Page

- Product
- Quantity
- Price
- Subtotal
- Remove

- Total
- Checkout

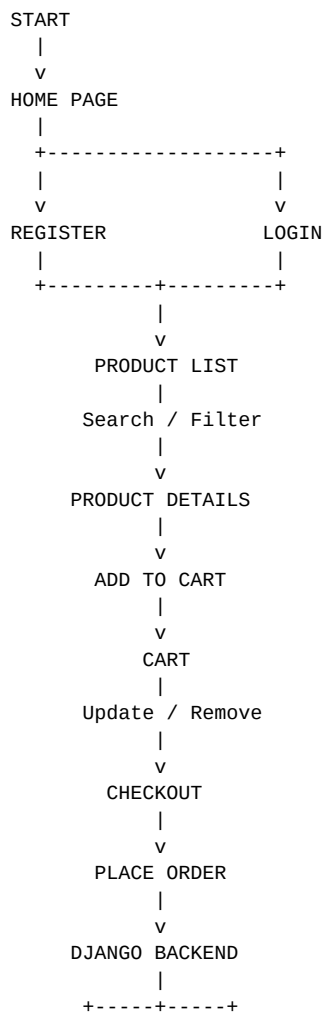
10.5 Checkout Page

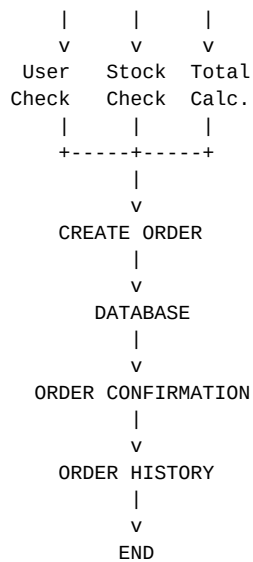
- Customer details
- Delivery address
- Order summary
- Total
- Place Order

10.6 Orders Page

- Order ID
- Date
- Products
- Total
- Status
- View Details

11. COMPLETE SYSTEM FLOW





12. SECURITY REQUIREMENTS

- Secure password hashing
- Authentication
- Authorization
- CSRF protection where applicable
- Input validation
- API access control
- Admin authorization
- Protection against unauthorized order access

13. HARDWARE REQUIREMENTS

Component	Minimum Requirement
Processor	Dual Core
RAM	4 GB
Storage	5 GB available
Network	Internet connection for dependency installation/development

14. SOFTWARE REQUIREMENTS

- Windows / Linux / macOS
- Python 3.x
- Django
- Django REST Framework
- SQLite / MySQL
- VS Code or another code editor

- Modern Web Browser
- Postman
- Git / GitHub

15. RECOMMENDED PROJECT DIRECTORY STRUCTURE

```
E-Commerce-Store/
|
+-- backend/
|   +-- manage.py
|   +-- ecommerce/
|       |   +-- settings.py
|       |   +-- urls.py
|       |   +-- wsgi.py
|       |
|       +-- users/
|       +-- products/
|       +-- cart/
|       +-- orders/
|
+-- frontend/
    +-- index.html
    +-- products.html
    +-- product-details.html
    +-- login.html
    +-- register.html
    +-- cart.html
    +-- checkout.html
    +-- orders.html
    |
    +-- css/
    |   +-- style.css
    |
    +-- js/
        +-- api.js
        +-- auth.js
        +-- products.js
        +-- cart.js
        +-- orders.js
```

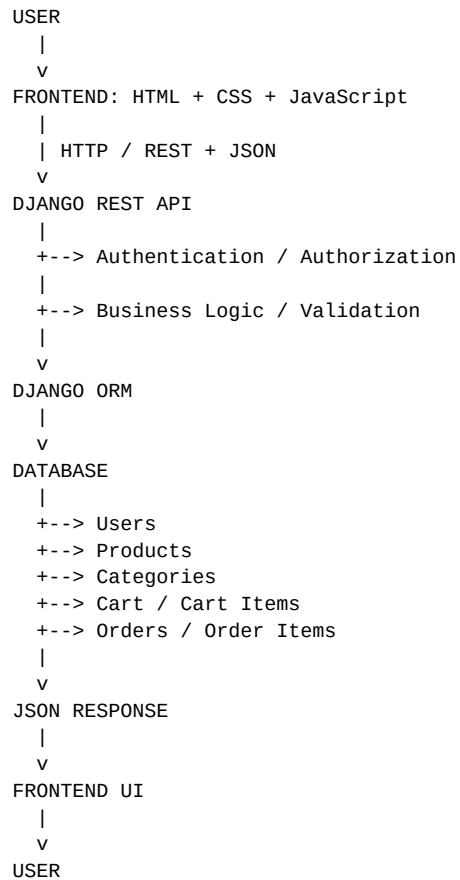
16. KEYWORDS FOR IMPLEMENTATION AND VIVA

Full-Stack Web Application	Client-Server Architecture	Three-Tier Architecture
REST API	HTTP Request/Response	JSON
Django	Django REST Framework	Django ORM
CRUD Operations	Authentication	Authorization
Password Hashing	Relational Database	Database Relationships
Business Logic	Input Validation	Error Handling
Shopping Cart	Checkout	Order Processing
Inventory / Stock Management	Order Status	Admin Module

17. FINAL ARCHITECTURE SUMMARY

The Simple E-Commerce Store uses a three-tier client-server architecture in which the HTML/CSS/JavaScript frontend communicates with a Django REST API through HTTP requests. Django handles authentication,

authorization, business logic and validation, and uses Django ORM to interact with the relational database. The backend returns JSON responses to the frontend, where JavaScript processes the response and updates the user interface.



Core Data Flow Statement

User → Frontend → HTTP Request → Django REST API → Business Logic → Django ORM → Database → JSON Response → Frontend → User